Greedy Algorithms

Greedy algorithms are a simple yet powerful design strategy used for solving optimization problems. The main idea is to build a solution step by step, always making the *locally optimal choice* at each step with the hope that these choices will lead to a globally optimal solution.

Greedy algorithms are intuitive and easy to implement. However, proving that they always produce an optimal result can be challenging. Sometimes they work perfectly, as in the *Fractional Knapsack* or *Huffman Coding*, and sometimes they only give an approximate solution, as in the *0–1 Knapsack*.

A greedy approach typically follows these steps:

1. Determine the rule for making a greedy choice (what is considered "best" at each step).
2. Process elements in a specific order, such as sorted by weight, profit, or start time.
3. Include an element only if it improves or keeps the solution valid.

Greedy methods are widely used in real-world problems such as:

- **Resource allocation:** selecting profitable items to maximize gain (for example, the Fractional Knapsack problem),
- **Scheduling:** assigning tasks or lectures to minimize overlaps,
- **Compression:** generating efficient codes to reduce data size (for example, Huffman Encoding).

Learning Outcomes

By the end of this tutorial, you will:

- Understand the concept of greedy choice and optimal substructure,
- Recognize when a problem can be solved optimally using a greedy strategy,
- Analyze the running time and correctness of simple greedy algorithms,
- Apply greedy techniques to problems like fractional knapsack, scheduling, and text compression.

Warm-up

Problem 1. Construct the Huffman tree of the following phrase: "data structure"

Problem 2. In the lecture, we learned that the FRACTIONAL KNAPSACK algorithm is only correct when it follows a specific greedy rule: always picking the item with the highest benefit-to-weight ratio.

However, if we change this greedy rule to instead:

1. always pick the item with the highest *benefit*, or
2. always pick the item with the *smallest weight*,

the algorithm does not necessarily give an optimal solution.

Your task is to show that even though these strategies are not correct in general, there are infinite classes of instances where each of them *does* give the optimal solution. This demonstrates that a few working examples are not enough to prove that an algorithm is correct in general.

Problem solving

Problem 3. Given vectors $a, b \in R^n$, consider the problem of finding a permutation a' and b' of a and b respectively minimizing their total difference

$$\sum_{1 \le i \le n} |a'_i - b'_i|.$$

Prove or disprove the correctness (i.e., that it always returns the optimal solution) of the following algorithm that greedily tries to pair up similar coordinates.

```
function GREEDY-MATCHING-V1(a, b)
    A ← multiset of values in a
    B ← multiset of values in b
    a' ← new empty list
    b' ← new empty list
    while A is not empty do
        x, y ← a pair in (A, B) minimizing |x − y|
        Append x to a'
        Append y to b'
        Remove x from A and y from B
    return (a', b')
```

Problem 4. Given vectors $a, b \in R^n$, consider the problem of finding a permutation a' and b' of a and b respectively that minimizes

$$\sum_{1 \le i \le n} |a'_i - b'_i|.$$

Prove or disprove the correctness (i.e., that it always returns the optimal solution) of the following algorithm that greedily tries to pair up similar coordinates.

```
function GREEDY-MATCHING-V2(a, b)
    A ← multiset of values in a
    B ← multiset of values in b
    a' ← new empty list
    b' ← new empty list
    while A is not empty do
        x ← smallest in A
        y ← smallest in B
        Append x to a'
        Append y to b'
        Remove x from A
        Remove y from B
    return (a', b')
```

Problem 5. You are given an alphabet $C = \{c_1, c_2, \ldots, c_k\}$ where symbol c_i appears with frequency $f_i = 2^i$ (so c_1 is least frequent and c_k is most frequent).

A Huffman tree is built by repeatedly merging the two currently least-frequent trees into a new parent whose weight is the sum of the two child weights.

Prove that in the Huffman tree T built from these frequencies, *every internal node has at least one child that is an external node* (a leaf corresponding to a single symbol).

Problem 6. Design a greedy algorithm for the following problem (see Figure 1): Given a set of n points $\{x_1, ..., x_n\}$ on the real line, determine the smallest set of unit-length intervals that contains all points. Unit-length interval is an interval where length = 1.

Figure 1: Covering points with unit intervals.

Problem 7. You are given n jobs $J = \{1, \ldots, n\}$. Each job j has a processing time $t_j > 0$ and a weight $w_j > 0$. All jobs are available at time 0. A schedule is a permutation σ of the jobs; jobs are processed non-preemptively in the order $\sigma(1), \sigma(2), \ldots, \sigma(n)$ on a single machine.

For a schedule σ, the completion time of job $\sigma(k)$ is

$$C_{\sigma(k)} = \sum_{r=1}^{k} t_{\sigma(r)}.$$

The objective is to minimize the weighted sum of completion times $\sum_{j=1}^{n} w_j C_j$.

a) Design a polynomial-time algorithm that outputs an optimal schedule. You may assume $t_j > 0$ and $w_j > 0$ for all j. If helpful, you may also assume no two ratios t_i/w_i are equal and comment on how to handle ties.

b) Prove that your algorithm is optimal using an exchange argument. Your proof should explicitly quantify how an adjacent swap of two out-of-order jobs changes the objective.

c) State the running time of your algorithm and justify it.

Challenge

This question is provided primarily to explore the boundaries of your problem-solving ability. It is intended as a stretch exercise and does not reflect the standard level of difficulty expected in the course.

Problem 8. A florist sells n different flowers, each with a price $p_1, p_2, \ldots, p_n$. There are k friends who decide to buy all the flowers together.

The florist wants to both *maximize his revenue* and *encourage more customers*. To achieve this, he introduces a rule:

- The first flower a customer buys costs its regular price.
- The second flower the same customer buys costs twice its regular price.
- The third flower costs three times its price, and so on.

In other words, if a friend has already bought x flowers, then the next flower they buy will cost $(x + 1)$ times its original price.

The group can choose the order in which the flowers are bought and decide which friend buys each flower. All flowers must be bought, and the goal is to minimize the total amount of money spent by the friends.

Example. Suppose there are $n = 5$ flowers with prices:

$$[2, 5, 6, 3, 8]$$

and $k = 2$ friends.

If the friends buy in the order shown above (2, 5, 6, 3, 8), alternating turns, then:

- Friend 1 buys 2 (×1) and 6 (×2), costing $2 + 12 = 14$.
- Friend 2 buys 5 (×1), 3 (×2), and 8 (×3), costing $5 + 6 + 24 = 35$.

The total cost is 49.

Your goal is to find a way to reduce this cost by choosing an effective order of purchases and deciding which friend should buy which flowers.

a) Describe a greedy strategy for deciding the order in which the flowers should be purchased and how to assign purchases to friends.

b) Explain why this strategy minimizes the total cost.

c) Give the time complexity of an implementation that follows this strategy.